Collaborative filtering & matrix factorization

Problem → steps → data issues → evaluation → overfitting → exam answers

Based on the requirements stated in this chat. Full chapter discussion and lecture verification remain pending. Examples are new illustrations, not recovered chat examples.

1. What are CF and MF? What problem do they solve?

• **Problem:** most users have rated or interacted with only a small part of the catalogue. Predict what each user may like and recommend useful unseen items.

• **Collaborative filtering (CF):** learn from patterns across users and items. You do not need a written description of every item to use basic CF.

• **User-based CF:** find users with similar rating patterns; use their feedback to make a recommendation. **Item-based CF:** find similarly rated items; use the user's feedback on those items.

• **Matrix factorization (MF):** a model-based method within CF. Learn a short list of numbers for each user and item; their match gives a predicted rating.

• **Relationship:** CF is the recommendation approach; MF is one way to implement it. They are not two compulsory consecutive algorithms.

2. One small example: from ratings to a prediction

• **Illustrative example, not recovered from the earlier chapter chat.** Movie ratings: A rated M1=5, M2=4, M3 unknown; B rated M1=5, M2=4, M3=5; C rated M1=1, M2=2, M3=1.

• **User-based reasoning:** A's known pattern resembles B's more than C's. B's rating is evidence that M3 may suit A. It is a recommendation signal, not proof A will give 5.

• **MF reasoning:** use all observed training ratings to learn user and movie factors. Factors describe patterns learned jointly; do not assume each one has a known label such as "action".

• **Prediction calculation:** suppose a trained model gives A's factors [1, 2] and M3's factors [2, 1]. Multiply matching positions and add: 1×2 + 2×1 = **4**. This is a basic predicted rating, not a probability.

• **What "learn" means:** the numbers above are supplied for calculation. In training, the model adjusts factor values so predictions better match known ratings; we do not manually assign these values.

3. MF steps: what happens, and why?

• **1 — Prepare:** keep user ID, item ID, feedback, and time if available. Remove erroneous records; decide how repeat interactions should be aggregated. Unknown ratings stay unknown.

• **2 — Split:** separate training, validation, and test interactions before fitting the model. Use earlier interactions to predict later ones when that matches the scenario.

• **3 — Choose a starting k:** k is how many factor values represent each user/item. Start with a manageable size; choose the final k using validation results.

• **4 — Initialize and predict:** start user/item factors with small values. For each known training rating, combine the user's and item's factors to predict it.

• **5 — Calculate error and update:** compare prediction with the actual rating; adjust factors to reduce the loss. With stochastic gradient descent (SGD), updates use individual training ratings or small batches. Repeat across the training data.

• **6 — Validate and stop:** check held-out performance, tune k and regularization, and keep the best validation model. Test once after choices are fixed.

• **7 — Recommend:** score eligible unseen items, order the scores, and recommend the highest-ranked items. A high predicted score is an estimate of preference, not a guarantee.

4. What is the loss? Explain it before writing symbols

• **Basic explicit-rating loss:** add up squared differences between actual and predicted **known training ratings**. Squaring makes errors positive and puts greater weight on large mistakes.

• **Small calculation:** actual ratings [5, 3], predictions [4, 4] give differences [1, −1]. Squared errors are [1, 1]; their total is **2**. This shows prediction-error loss, not the complete regularized objective.

• **Regularized loss:** prediction-error loss **plus a penalty for large factor values**. The penalty discourages fitting every accidental detail. Its strength is tuned on validation data.

• **Training loss vs evaluation:** the loss guides updates on training data. A validation/test metric measures performance on held-out observations. Low training loss alone does not show good recommendations.

• **Implicit feedback:** a click/watch is evidence of interaction, not an explicit rating. Do not blindly reuse an explicit-rating objective: the handling of unobserved pairs must match the task.

5. Data problems: problem → remedy → reason

• **Sparse matrix:** most user–item pairs are missing. Similarity estimates may have little shared evidence; factor estimates may also be weak. MF shares patterns across ratings, but does not create information from nothing.

• **Practical response:** store observed user–item–rating records rather than a huge dense table; use a small enough factor model and regularization. Sparse storage saves memory; it does not itself improve prediction quality.

• **Do not fill missing ratings with 0:** that tells the model an unknown item was disliked. For explicit-rating MF, fit known ratings and predict the missing ones. Collect more relevant feedback where possible.

• **Few ratings vs no history:** an existing user with a little history may still get an estimate from shared patterns. A completely new user/item has no learned factors from interactions: this is **cold start**.

• **Cold-start response:** ask a new user for a few preferences; use a popularity/average fallback initially; or use item/user information with a content-based or hybrid approach. Explain that extra information is being added to basic CF.

• **Different rating habits:** one user may rate everything generously. If this is a problem, account for user/item baseline

tendencies so systematic generosity is not mistaken for special preference.

• **Clicks and repeated events:** no click is not automatically dislike; the item may never have been shown. Keep the meaning of feedback clear and do not let accidental duplicates pretend to be independent evidence.

6. How do I evaluate it?

• **Split roles:** training learns factors; validation chooses settings/stopping point; test estimates final performance. Do not choose k using test results.

• **Rating prediction:** compare predicted ratings with held-out actual ratings. MAE is average absolute error; RMSE squares errors, averages them, then takes the square root. Lower is better; RMSE reacts more to large errors.

• **Calculation:** errors [1, −1] give MAE=(1+1)/2=**1**; RMSE=sqrt((1+1)/2)=**1**, in rating units.

• **Top-item recommendation:** ask whether relevant items appear near the top. Precision@N = relevant items among the N recommendations / N. Recall@N = relevant items retrieved / all held-out relevant items. State what counts as relevant.

• **Ranking calculation:** recommend 5 items; 3 are relevant; the user has 6 relevant held-out items. Precision@5=3/5=**0.60**; Recall@5=3/6=**0.50**. Here N counts recommendations; k counts factors.

• **Check fairness of comparison:** use the same split and candidate rules for competing models; compare against a simple average/popularity baseline. Evaluate new-user cases separately if those are the intended task.

• **Avoid leakage:** do not train on future ratings and then claim to predict the past. Do not compute fitted rating averages or model parameters from held-out feedback.

7. Overfitting: recognize it and choose a remedy

• **Recognize:** training error falls while validation error rises. The model is fitting training-specific detail rather than improving on unseen feedback.

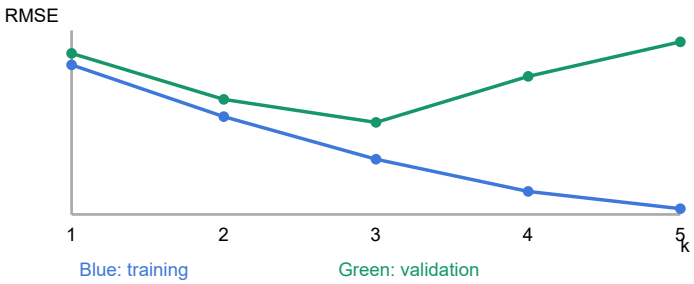
• **Reduce k:** fewer factor values mean less capacity to fit noise. Too few can underfit, so validate rather than assuming “smaller is always better”.

• **Add/increase regularization:** penalize large factor values so a few observations cannot pull estimates too far. Too much can underfit; select the strength using validation.

• **Early stopping:** stop training when validation performance stops improving and retain the best checkpoint. More training passes are not automatically better.

• **Improve useful evidence:** collect genuine additional feedback and remove erroneous/duplicated records. Copying the same rows is not new information.

• **Example choosing k:** k=[1,2,3,4,5]; training RMSE=[1.30,0.85,0.48,0.20,0.05]; validation RMSE=[1.40,1.00,0.80,1.20,1.50]. Choose **k=3** here, not k=5. Values illustrate the decision; they are not lecture results.



8. Turn the scenario into an answer

• **Suggested MF, very sparse ratings:** “MF is suitable for learning shared user/item patterns, but sparse feedback makes estimates uncertain. I would tune a smaller factor count and regularization on validation data; use a fallback when history is insufficient.”

• **Training improves, validation worsens:** “This indicates overfitting. I would reduce model capacity or increase regularization, stop at the best validation point, and check the final model on untouched test data.”

• **Unknowns replaced by zero:** “Missing is not the same as dislike. These zeros introduce false negative ratings. I would preserve missingness and use an objective appropriate for the observed feedback.”

• **Good RMSE, weak recommendation list:** “Rating accuracy and ranking quality are different. I would evaluate relevant items in the top-N list and choose settings consistent with the recommendation objective.”

• **Answer order:** identify the task → select/assess the model → identify the specific problem → propose the change → explain why → say how to evaluate the change.